Full Stack Development Week 04

Topics covered:

- Form handling and validation
- Conditional rendering
- Login Logout
- GitHub (again)

Activity 1: Assignment 1 Requirements Gathering Workshop 1 of 2 (40 minutes)

By now, you should already find a programming partner and form a team. Remember to register your names on Canvas and create the matching team name on GitHub Classroom A1 repo.

Your tutor is your client. In most companies, if they adopted the agile methodologies like Scrum, your client might be represented by a Product Owner, which is the person responsible for translating business requirements into workable chunks from the end user perspective called 'user stories' and provide feedback each time a user story is done. Each user story has the following format:

As a _____ (the user role, who is using the app)

I want to _____ (the task, what the user can specifically do using the app)

So that _____ (the goal or achievement, why do they want to do that)

Example of good user story:

"As a hirer I want to browse the available venues, so that I can view which venues are suitable for my event".

Each user story is associated with at least 1 quality standard called 'acceptance criteria' which defines the Product Owner's expectations in terms of meeting the functional and non-functional requirements.

Examples of good acceptance criteria are as clear as follows:

"When the vendor pressed the log out button, the will go to an exit page saying, 'See you next time!'"

"When the hirer selected the top 3 venue candidates at the top part of the screen, the three items would appear in the middle and ranked according to the order of selection"

However, some acceptance criteria could be ambiguous or not very well defined. As a programming pair, your task is to navigate through some ambiguities, seek clarifications, make assumptions, make design decisions with partial information, work with your product owner to help sharpen the acceptance criteria, and still deliver the promised features on time.

In this assignment, the acceptance criteria will be linked to rubric and grades.

Your tutor will give you the briefs in Week 4 and Week 5 about what to prioritise and will expect that most of the intended features will be done and ready to test in Week 6. Please make sure you attend these pracs or if you happen to miss them, please ensure that you are in good coordination with your partner who will attend it.

Activity 2: Form handling and validation (20 mins)

Create a simple login form using React with the following requirements:

1. Create a form with email and password fields
2. Implement validation:
    - Email must be a valid email format (e.g., user@example.com)
    - Password must be at least 8 characters long and contain at least one uppercase letter, one lowercase letter, and one number
3. Display validation errors to the user
4. Handle form submission

Discussion: Two Approaches to Form Validation

- Using built in HTML validation
- Using Javascript validation, pros / cons.

Activity 3: Conditional rendering (20 mins)

For this activity, let's use conditional rendering to extend our existing code from Activity 2. This time to show a success message when the form has been successfully submitted.

Activity 4: Login, logout and make a post. (20 mins)

> Note: Successful completion of this activity will help with the login-logout feature in Assessment 1.

Let's revise Lab Sheet 3 again. This time adding the ability to make a post. Final workflow:

- User logs in
- User can see that they are logged in
- Forum link now displays in the header for a logged in user (conditional rendering)
- Clicking on the Forum link takes us to the forum page
- User can make a post which gets displayed.

Activity 5: Course admin (remaining time)

Please use the remaining time to organise your repos, continue with last week's tasks if

you have not completed all of them, and work with your partner and 'product owner' to

clarify assignment requirements if in doubt.

Reference Notes for managing GitHub repo

This exercise assumes you have joined the rmit-fsd-2026-s1 GitHub organisation, the relevant GitHub Class Room repos and have Git installed.

Steps:

1. Create a Next.js project:

   ```
   npx create-next-app@latest s1234567-test
   ```

   (Replace `s1234567`with your student ID.)

   When prompted, select:

   - TypeScript → Yes
   - ESLint → Yes
   - Tailwind CSS → Yes
   - src/ directory → Yes
   - App Router → No
   - Import alias → No

2. Open the project folder in Visual Studio Code.

3. In VS Code, open the Source Control tab (Ctrl+Shift+G or click the branch icon).

4. Click "Publish to GitHub" to create a new Git repository.

5. Type an initial commit message like "Initial commit" in the message field.

6. Click the checkmark icon (✓) above the message to commit the changes.

7. Click "Publish Branch" to push to GitHub.

   - Select "Private repository" when prompted
   - Choose a repository name
   - Click "Publish"

You can find more instructions on getting started with Git in Visual Studio Code at the following URL: https://code.visualstudio.com/docs/sourcecontrol/intro-to-git.

Committing and Pushing Changes:

1. Make changes to your code files.
2. In VS Code's Source Control tab:
    - Changed files appear under "Changes"
    - Click the + icon next to files to stage them
    - Type a descriptive commit message
    - Click the checkmark (✓) to commit
3. Push commits to GitHub:
    - Click the "..." menu in Source Control
    - Select "Push" or use Ctrl+Shift+P
    - Or click the sync icon in the status bar
4. Best practices for commits:
    - Write clear commit messages
    - Commit related changes together
    - Commit often
    - Don't commit broken code

Example commit messages:

- "Add login form component"
- "Fix navigation bug in header"
- "Update styling for mobile responsiveness"
- "Implement user authentication"
- "Add form validation to registration"

Using Git from the Command Line:

1. Initialize a new Git repository:

```
git init
```

2. Add your GitHub repository as a remote:

```
git remote add origin https://github.com/username/repository.git
```

3. Stage all files:

```
git add .
```

4. Create initial commit:

```
git commit -m "Initial commit"
```

5. Push to GitHub:

```
git push -u origin main
```

For ongoing changes:

1. Check status of changes:

```
git status
```

2. Stage modified files:

```
git add <filename>
# Or stage all changes: git add .
```

3. Commit changes:

```
git commit -m "Your commit message"
```

4. Push to GitHub:

```
git push
```

Common Git commands:

- `git pull`- Get latest changes from remote
- `git branch`- List branches
- `git checkout -b branch-name`- Create and switch to new branch
- `git merge branch-name`- Merge branch into current branch
- `git log`- View commit history